CHOP Workshop — AISQL + Snowflake Intelligence

SE Reference Guide & Workshop Agenda

2 hours | 2 roles: Analysts + Scientists | 3 projects: SI • ML • Cost Governance | Audience: CHOP Data & Analytics team

Pre-Workshop

Pre-Workshop Checklist (1 week before)

Admin completes these steps before participants can run the readiness check. Run scripts in order from `sql/infrastructure_setup_guide/` in the GitHub repo.

Admin Setup (scripts 00–12)

1. Admin runs `00_pre_flight_column_check.sql` — pastes all 3 result sets to SE. SE fixes column names in `01_si_infrastructure.sql`, pushes corrected script.
2. Run scripts 01–12 in order (skip 02, 13). See admin-routine memory for full sequence.
3. After script 10: run standalone grant: `GRANT READ ON GIT REPOSITORY ... TO ROLE ML_ENGINEER;`

Participant Readiness Check

1. SE sends participants `sql/CHOP_participant_readiness_check.sql` (in Git repo).
2. Analysts run Section A, Scientists run Section B — each takes <30 seconds.
3. Participants screenshot results and send to SE by EOD the day before.
4. SE resolves any FAIL rows with admin. Zero surprises on workshop day.

Quick Reference — Roles & Warehouses

Role	Audience	Warehouse	Data access
`CHOP_SNOW_INTELLIGENCE`	Data Analysts	`CHOP_SNOW_INTELLIGENCE_WH`	SI_CHOP views, semantic views, search services, agent
`ML_ENGINEER`	Data Scientists	`HEALTHCARE_ML_WH`	HEALTHCARE_ML tables, feature store, model registry, Git repo, agent
`AI_EXPLORER`	Exploratory users	either	$50/month Cortex spend budget enforced hourly
`AI_DATA_SCIENCE`	Power users	either	$100/month Cortex spend budget enforced hourly

Overview Session Overview — 2 hours

Time	Block	Format	Who drives
0:00–0:05	Welcome + env check (Cell 1 of notebook)	Everyone runs Cell 1 — confirms readiness	SE facilitates
0:05–0:30	SE walkthrough — AISQL concepts	SE presents; participants follow on screen	SE
0:30–1:00	Hands-on AISQL exercises (Cells 2–4)	Participants run notebook; SE answers questions	Participants
0:55–1:00	Cortex Code bridge (Cell 5)	SE uses coco CLI live to generate agent spec	SE + coco
1:00–1:10	**Break (10 min)**		
1:10–1:20	coco output vs pre-built agent — side by side	SE compares coco-generated SQL to `06_si_agent_creation.sql`	SE
1:20–1:40	Live agent demo (Cell 6 call sheet)	Each analyst submits 1 question; SE types it; all see the response	Participants + SE
1:40–2:00	Architecture + Q&A	Whiteboard walkthrough; scientists bridge to ML readmission agent concept	SE

AISQL Block Hands-On Exercises — Cells 2–4 (notebook)

Both roles can participate in all three exercises. Each exercise has: (A) an inline text version that works for everyone, and (B) a role-specific table version. The notebook auto-detects the role and runs the right version.

1 AI_EXTRACT — Extraction patterns & transformation workflows

Cell 2 ~10 min

Concept: AI_EXTRACT turns unstructured clinical text into a structured JSON object. You define what fields to pull — the LLM finds them. Fields not present in the text are simply omitted.

Part A — inline text (both roles run this first):

```
SELECT SNOWFLAKE.CORTEX.AI_EXTRACT(
  'Administer methotrexate 25mg subcutaneously once weekly.
   Monitor LFTs monthly. Hold if WBC < 3.0.',
  ['medication_name', 'dose', 'route', 'frequency',
```

```
    'monitoring_instructions', 'hold_condition']
) AS extracted_entities;
```

ANALYSTS — PART B

Extract from `ADMINISTRATIONDIRECTIONS` column in `V_PHARMACY_ALLMEDICALSCRIPTS`. Real CHOP pharmacy SIG text. Compare AI output to what your SIG parsing already captures.

SCIENTISTS — PART B

Build a clinical narrative string from `PRIMARY_DIAGNOSIS`, `DISCHARGE_DISPOSITION`, `LENGTH_OF_STAY` columns, then extract clinical summary entities. Shows: structured data → narrative → extraction pattern.

SE talking point: This is the same UDF (`EXTRACT_PRESCRIPTION_ENTITIES`) that the SI agent calls as a tool. In the agent, analysts trigger this by just asking a question — no SQL required.

2 AI_CLASSIFY — Guardrails & quality evaluation

Cell 3 · ~8 min

Concept: Classify free text against a controlled set of labels you define. Use as a guardrail to catch mis-labeled or ambiguous values before they enter downstream tables.

Part A — inline text (both roles):

```
SELECT SNOWFLAKE.CORTEX.AI_CLASSIFY(
  'infuse 500ml normal saline over 4 hours through peripheral IV line',
  ['oral', 'intravenous', 'intramuscular', 'subcutaneous',
   'topical', 'inhaled', 'other']
) AS route_classification;
```

ANALYSTS — PART B

Classify `ADMINISTRATIONDIRECTIONS` and compare to stored `ADMINROUTE` column. Where they disagree = your data quality signal. How often does the stored value mismatch?

SCIENTISTS — PART B

Classify `DISCHARGE_DISPOSITION` (HOME, SNF, AMA...) into readmission risk tiers and compare to actual `READMITTED_30D` outcome. Discussion: LLM classification vs hand-engineered `DISPOSITION_RISK_SCORE` — when does each win?

3 AI_FILTER + COMPLETE — Quality gates & transformation

Cell 4 · ~7 min

Concept: `AI_FILTER` returns True/False — use it in a `WHERE` clause to gate low-quality records before extraction. `COMPLETE` is the transformation function — rewrite, standardize, enrich.

Both roles run the same cells (no table access needed):

```
-- Gate: only extract from complete, actionable SIG text
SELECT SNOWFLAKE.CORTEX.AI_FILTER(
```

```
  'take as needed',
  'A complete prescription with dose, frequency, and route'
) AS passes_quality_gate;
-- Returns FALSE → discard this record

-- Transform: standardize an informal clinical note
SELECT SNOWFLAKE.CORTEX.COMPLETE(
  'llama3.1-70b',
  'Standardize to clinical format with dose, route, frequency,
   duration, monitoring: "give 2 tabs twice a day with food for 2 weeks, watch liver"'
) AS standardized_note;
```

SE talking point: The pipeline pattern is: AI_FILTER → AI_EXTRACT on passing records only → store structured output. Quality gating before extraction reduces LLM cost and prevents garbage data from entering your feature store or semantic views.

coco Bridge — Cortex Code Moment — Cell 5 (SE-led, ~5 min before break)

> **Goal:** Connect the AISQL exercises to the SI agent demo. SE types a prompt into the coco CLI live — coco generates the `CREATE AGENT` SQL spec. Then SE reveals the pre-built agent is essentially the same spec, already deployed.

coco prompt to type live:

```
I have these objects in SI_CHOP.CHOP_SNOW_INTELLIGENCE:

- PRESCRIPTION_ORDERS_SV: semantic view on pharmacy prescriptions
  (facts: script_number, costs, quantities; dimensions: drug, route, SIG)
- MEDICATION_ORDERS_SV: semantic view on Epic medication orders
- DRUG_CATALOG_SEARCH: Cortex Search on drug catalog (name, NDC, category)
- PRESCRIPTION_DIRECTIONS_SEARCH: Cortex Search on SIG free text
- EXTRACT_PRESCRIPTION_ENTITIES(VARCHAR): UDF that calls AI_EXTRACT on SIG
- GENERATE_STREAMLIT_APP(VARCHAR): procedure for Streamlit dashboards

Generate the SQL to CREATE a Cortex Agent that uses all 6 as tools
and can answer pharmacy questions in natural language.
```

After coco generates the spec:

- ✓ Point out the `tool_spec` block structure — same 4 tool types used in exercises
- ✓ Show `EXTRACT_PRESCRIPTION_ENTITIES` in the spec as a generic function tool — this is the same UDF participants called in Cell 2
- ✓ Open `06_si_agent_creation.sql` side-by-side — "this is exactly what we pre-deployed"
- ✓ Say: "After the break, let's query it."

SI Block

Snowflake Intelligence Demo — 60 min post-break

Time	Activity	SE Notes
+0 min	coco output vs deployed agent	Side-by-side of coco SQL and `06_si_agent_creation.sql`. Show they're structurally identical. "coco wrote this in 30 seconds."
+10 min	Live agent questions (Cell 6 call sheet)	Each analyst picks one of the 5 call-sheet questions. SE types it in Snowsight Agents. Show the tool trace — which tool was invoked.
+30 min	Architecture walkthrough	Walk the diagram below. Emphasize: views cap cost ($13-15/month Cortex Search vs $150 without the filter). Governance: semantic views enforce business logic centrally.
+50 min	Scientists bridge — readmission agent concept	"How would you build this for your ML use case?" Walk through: semantic view on ADMISSIONS + FEATURE_STORE, generic function calling READMISSION_PREDICTOR model, natural language question: 'Which patients admitted last week are high risk?'

Architecture diagram (draw on whiteboard or reference this):

```
User question (natural language) | ▼ Cortex Agent (orchestrator LLM) |
┌─────┴──────────────────────────────────────────────┐ | | | ▼ ▼ ▼ Cortex Analyst Cortex
Search Generic Function (text-to-SQL) (semantic fuzzy (EXTRACT_PRESCRIPTION_ENTITIES
lookup) = AI_EXTRACT from Cell 2) | | ▼ ▼ PRESCRIPTION_ORDERS_SV DRUG_CATALOG_SEARCH
MEDICATION_ORDERS_SV PRESCRIPTION_DIRECTIONS_SEARCH | ▼ V_PHARMACY_ALLMEDICALSCRIPTS ←
12-month filter + 50K row cap | (cost control: ~$13-15/month) ▼
PROD.LAKE_HDMS.DS_PHARMACY_ALLMEDICALSCRIPTS (source of truth)
```

SI call sheet — 5 questions for the live demo:

#	Question	Agent tool invoked
1	*"What are the top 10 most prescribed drugs by script count?"*	Cortex Analyst → PRESCRIPTION_ORDERS_SV
2	*"Show me all IV-route prescriptions from the last 30 days"*	Cortex Analyst → PRESCRIPTION_ORDERS_SV
3	*"Find drugs related to amoxicillin in the drug catalog"*	Cortex Search → DRUG_CATALOG_SEARCH
4	*"Extract the medication name and dose from: 'Give 25mg MTX SQ weekly, hold if WBC < 3'"*	Generic function → EXTRACT_PRESCRIPTION_ENTITIES
5	*"What therapeutic classes have the most medication orders?"*	Cortex Analyst → MEDICATION_ORDERS_SV

Assets

Files Created / Modified for This Workshop

File	Type	Purpose
`healthcare-readmission-ml/notebooks/workshop_aisql_si_exercises.ipynb`	New	Participant notebook - Cells 1–6. Env check, exercises, coco bridge SI call sheet. Auto-detects role.
`sql/CHOP_participant_readiness_check.sql`	New	Send to participants 1 week before. Section = Analysts, Section B Scientists. PASS/FAIL output, <30 seconds.
`sql/infrastructure_setup_guide/06_si_agent_creation.sql`	Modified	Added: `GRANT USAGE ON AGENT ... TO RO ML_ENGINEER` so scientists can access the agent.
`sql/infrastructure_setup_guide/01_si_infrastructure.sql`	Modified	6 views: 12-month rolling filter + 50K row cap. Cost control for Cortex Search (~$13-15/month).
`sql/infrastructure_setup_guide/04_si_cortex_search.sql`	Modified	TARGET_LAG changed from `'1 day'` to `'1 days'` on both searcl services.
`sql/unified_admin_setup/02_roles_and_grants.sql`	Modified	Git repo grant commented out (move to script 10 where the repo object actually exists).
`sql/infrastructure_setup_guide/00_pre_flight_column_check.sql`	New	3-part admin diagnost INFORMATION_SCHE column dump, date column profiler, join k validation with anchor join strategy.